Inference Engineering Week Task

Tejal Pagar
AIML TEP Cohort

October 18, 2026

Question 1: What's the difference between latency and throughput in model inference, which factor will influence model's performance more and how?

While reading about this I understood that both of them measure speed, but from two different sides.

Latency is the time the model takes to give an answer for one request. For example, if I send one sentence to a model and get the output back after 50 ms, then the latency is 50 ms. This is what a single user feels when he is waiting for a reply (like in a chatbot).

Throughput is how many requests (or tokens) the model can finish in a given time, like samples per second or tokens per second. It is more about the whole system than one user.

$$\text{Throughput} = \frac{\text{number of samples processed}}{\text{total time taken}}$$

A simple example: a bus vs a car. A car reaches the destination faster (low latency) but carries only 4 people. A bus is slower but carries 50 people, so more people reach per hour (high throughput).

Which factor influences performance more? From what I learned, **batch size** is the biggest factor here. When we increase the batch size, the hardware does more work in one go, so the total time for the batch goes up a little but the time per sample goes down. So throughput improves, but each individual request has to wait for the whole batch, so latency becomes worse. Because of this there is always a trade-off. Other things like model size, hardware (CPU/GPU), sequence length and precision (fp32 vs int8) also matter, but I think batch size is the one that directly controls the latency–throughput trade-off. If an app needs fast replies for one user (chatbot) we keep batch small, and if it is an offline job (like processing a million reviews) we use big batches.

Question 2: Explain what KV cache is and how it helps with model's generation

In a transformer, each token in the attention layer is converted into three things: Query (Q), Key (K) and Value (V). The attention is calculated as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

When a model generates text, it makes one token at a time (autoregressive). To generate token number $t + 1$, it needs the keys and values of all the previous t tokens. Without any caching, the model would calculate K and V for all the old tokens again and again at every step, which is a big waste because the old tokens do not change.

KV cache just means we store (cache) the K and V vectors of the previous tokens in memory. At every new step we only compute Q, K, V for the *new* token, add the new K and V to the cache, and use the cached ones for attention.

How it helps:

- Saves a lot of repeated computation, so generation is much faster, especially for long outputs.
- Without cache the work for each step grows with the whole length, with cache each step only has the new token's work (plus reading the cache).

The downside (the trade-off) is that the cache takes memory, and it grows with the sequence length, number of layers and batch size. For long contexts this memory can become a real problem, which is why people try to manage it more smartly (for example PagedAttention in vLLM).

Question 3: What is quantization, and what's the trade-off between int8/int4 quantization and model accuracy? Also give an example with calculations and formula of how quantization can be performed

What is it? Quantization means storing the model's numbers (weights, and sometimes activations) using fewer bits. Normally weights are 32-bit floats (fp32). In quantization we convert them to 8-bit integers (int8) or 4-bit integers (int4). So the model becomes smaller and usually faster, because integer maths and less memory movement is cheaper.

Memory example. If a model has 7 billion parameters:

- fp32: $7 \times 10^9 \times 4$ bytes = 28 GB
- int8: $7 \times 10^9 \times 1$ byte = 7 GB
- int4: $7 \times 10^9 \times 0.5$ byte = 3.5 GB

Trade-off with accuracy. When we squeeze a float into a small integer we lose some precision (rounding error). int8 has 256 levels so the error is usually small and the accuracy drop is often tiny. int4 has only 16 levels, so the rounding error is bigger and

the accuracy can drop more, so people use extra tricks to reduce this loss. So basically: fewer bits ⇒ smaller and faster model, but more chance of losing accuracy.

How quantization is done (symmetric formula).

$$s = \frac{\max(|w|)}{q_{max}}, \qquad q = \text{round}\left(\frac{w}{s}\right), \qquad \hat{w} = q \times s$$

Here s is the scale, q is the quantized integer, and $\hat{w}$ is the value we get back after dequantizing. For int8, $q_{max} = 127$ and for int4, $q_{max} = 7$ (range -7 to 7).

Example. Take these 5 weights: $w = [-1.2,\ 0.4,\ 0.9,\ -0.3,\ 2.0]$. Here $\max(|w|) = 2.0$.

int8:

$$s = \frac{2.0}{127} = 0.015748$$

$$q = \text{round}(w/s) = [-76,\ 25,\ 57,\ -19,\ 127]$$

For example, $-1.2/0.015748 = -76.2 \approx -76$. Now dequantize: $\hat{w} = q \times s = [-1.1969,\ 0.3937,\ 0.8976,\ -$

int4:

$$s = \frac{2.0}{7} = 0.2857$$

$$q = [-4,\ 1,\ 3,\ -1,\ 7]$$

Dequantized: $\hat{w} = [-1.1429,\ 0.2857,\ 0.8571,\ -0.2857,\ 2.0000]$.

	-1.2	0.4	0.9	-0.3	2.0
Original w	-1.2	0.4	0.9	-0.3	2.0
int8 $\hat{w}$	-1.1969	0.3937	0.8976	-0.2992	2.0000
int8 error	0.0031	0.0063	0.0024	0.0008	0
int4 $\hat{w}$	-1.1429	0.2857	0.8571	-0.2857	2.0000
int4 error	0.0571	0.1143	0.0429	0.0143	0

So from the table we can clearly see that the int4 errors (up to about 0.11) are much bigger than the int8 errors (up to about 0.006). This is the accuracy vs size trade-off in a small example.

Question 4: Name one real inference serving framework (like vLLM or TensorRT-LLM) and briefly describe one optimization it does

One real framework is **vLLM**. It is an open-source library used for serving large language models fast.

One optimization it does is **PagedAttention**. As I explained in the KV cache answer, the cache takes a lot of memory and it grows as the text grows. In older systems the memory for the KV cache was reserved as one big continuous block for each request, so a lot of memory got wasted (fragmentation) because the final length of the output is not known in advance. PagedAttention takes the idea of paging from operating systems: it splits the KV cache into small fixed-size blocks which need not be next to each other in memory, and gives blocks to a request only when needed. Because of this, memory is wasted much less, so more requests can fit in one batch and the throughput goes up.